Performance Analysis of Sparse Matrix-Vector Multiplication (SpMV)

Michael Bernasconi
ID Student: 267681
michael.bernasconi@studenti.unitn.it
https://github.com/Michael-Bernasconi/Sparse-Matrix-Vector-Multiplication
Department of Information Engineering and Computer Science
University of Trento

***Abstract*——-**
***Index Terms*—component, formatting, style, styling, insert**

I. Introduction

The Sparse Matrix-Vector Multiplication (SpMV) operation, defined as $y = Ax$, constitutes a critical computational kernel for High-Performance Computing. Due to its low arithmetic intensity, quantifiable as $2 \times nnz$ total operations, SpMV is an inherently memory-bound operation. Its efficiency is further hindered by the irregular nature of memory accesses, which strictly depend on the matrix's sparsity structure. This study analyzes the impact of storage formats and parallelization strategies on the NVIDIA Ampere architecture, comparing custom CUDA implementations (COO, CSR, and CSR-Vector) with the NVIDIA cuSPARSE library.
The investigation evaluates performance through key metrics: kernel execution time, throughput (GFLOPS), and effective bandwidth (BW). This study correlates matrix structural characteristics with hardware performance, offering an experimental guide for selecting the optimal format and kernel to mitigate computational bottlenecks.

II. Methodology

This section illustrates the test environment, the examined matrix formats, the CPU and GPU implementations, the rigorous methodology adopted for performance measurement, and the methods to validate their mathematical correctness.

A. Hardware and Software Environment

The experiments were conducted on a compute node, whose main specifications are summarized in Table I.

TABLE I
Hardware Environment Specifications

Feature	Host (CPU)	Device (GPU)
Model	Intel Xeon Silver 4309Y	NVIDIA A30
Architecture	Ice Lake (x86_64)	Ampere (Compute Cap. 8.0)
Cores / SM	16 Cores / 32 Threads (2 Sock.)	56 SM (3584 CUDA Cores)
Clock Frequency	2.80 GHz (Base) / 3.60 GHz	1.44 GHz (Boost)
FP32 Performance	1.84 TFLOPS	10.3 TFLOPS
Memory Bandwidth	102.4 GB/s	933.1 GB/s (HBM2)
Total Global Memory	N/A	24 GB
L1 Cache	768 KiB Data / 512 KiB Instr.	192 KiB per SM (Unified)
L2 Cache	20 MiB	24 MiB (Shared LLC)
L3 Cache	24 MiB (Shared LLC)	N/A (L2 acts as LLC)
Shared Memory	N/A	48-100 KiB / SM
Thread Limits	2 Threads/Core	1024 th/block (2048 th/SM)
Warp Size	N/A	32

B. CPU Baseline and GPU Implementations

CPU Baseline: The reference computation on the CPU was handled through a parallel implementation based on OpenMP. This computational phase is not intended as a performance benchmark, but is used for the validation and correctness check on the results subsequently computed in parallel by the GPU.
In GPU computation, SpMV performance critically depends on the interaction between the memory layout of the sparse matrix and the adopted execution model. In order to analyze this dual aspect (data structure and computational logic), our investigation focuses on the following approaches:

1. COO Format and Base Kernel

The Coordinate (COO) format explicitly stores the sparse matrix through three arrays of size equal to the number of non-zero elements (NNZ): row indices, column indices, and values.

Algorithm 1: SpMV Kernel - GPU Base COO

```
i ← blockIdx.x × blockDim.x + threadIdx.x
if i < nnz then
    atomicAdd(&y[rows[i]], vals[i] × x[cols[i]])
end if
```

The Base COO kernel adopts a direct mapping strategy: it launches a single thread for each non-zero element. This approach distributes an excellent load balance among the multiprocessors, being totally insensitive to the distribution of elements in the rows. However, since multiple threads can process elements belonging to the same row simultaneously, it forces the massive use of atomic locks (`atomicAdd`) to resolve *race conditions* when writing to the output vector y, limiting overall performance.

2. CSR Format and Base Kernel (Scalar)

The Compressed Sparse Row (CSR) format optimizes the memory footprint of COO by compressing the row index array into a `row_ptr` vector of size $M+1$ (where M is the number of rows), keeping the column and value vectors explicit.

Algorithm 2: SpMV Kernel - GPU Base CSR

```
row ← blockIdx.x × blockDim.x + threadIdx.x
if row < M then
    sum ← 0.0
    row_start ← row_ptr[row]
    row_end ← row_ptr[row + 1]
    for j ← row_start to row_end − 1 do
        sum ← sum + vals[j] × x[col_idx[j]]
    end for
    y[row] ← sum
end if
```

The Base CSR kernel maps a single thread to a single row of the matrix. This allows the sequential accumulation of products on a local register (`sum`), completely eliminating atomic write operations. Its critical disadvantage emerges with highly irregular matrices: rows of different lengths cause a severe temporal imbalance among the threads of the same warp and generate non-coalesced global memory access patterns.

3. Optimized CSR-Vector Kernel

To overcome the inefficiencies of Base CSR without altering the memory format, the CSR-Vector kernel redefines the execution granularity. Instead of assigning one thread per row, it maps an entire *warp* (32 threads) to process a single row. The optimization is divided into three phases:

1) Coalesced Access: The threads within the warp use their internal index (`lane_id`) to access the row elements in a strided manner. This guarantees perfectly aligned global memory transactions, maximizing bandwidth utilization.
2) Fast Accumulation: Partial results are stored in *Shared Memory* to minimize local read/write latency.
3) Parallel Reduction: Once the row traversal is complete, the warp threads perform a tree reduction to collapse the sums. The *leader* thread will finally write the accumulated value to the output vector in a unique manner.

Finally, as an absolute *baseline* to measure the effectiveness of the custom optimizations, all experiments were compared with the performance of the **cuSPARSE** library from the NVIDIA suite.

C. Measurement Methodology

The accuracy of the benchmark is guaranteed by the following methodological choices:

- *Output Consistency:* To prevent accumulations of incorrect values between iterations (a critical phenomenon in the COO format which uses the `atomicAdd` function), the destination vector is explicitly zeroed out before each kernel launch.
- *Variability Mitigation and Measurement:* The pure computation time (*Kernel Time*) is evaluated, ignoring the initial I/O (parsing) and memory allocation costs. To obtain stable measurements, each test is divided into two phases:
  - Warm-up (20 iterations): Necessary to stabilize the hardware and bring the GPU frequencies up to speed.
  - Benchmark (500 iterations): Actual performance measurement.

  The entire procedure is repeated for 5 independent *runs*. The average time recorded during the benchmark phases (accompanied by the standard deviation to prove its stability) becomes the base parameter upon which all final performance metrics are calculated: Kernel Time, Throughput (GFLOPS), and Effective Bandwidth (BW).

D. Performance Metrics

The implementations are evaluated using the following equations:

- Kernel Time (T_{avg}): Recorded using `cudaEvents` to sample solely the lifetime time window of the kernel on the VRAM, evading CPU-bound latencies.
- Computational Throughput (GFLOPS): The standard recognizes 2 float operations (FMA: one multiplication and one addition) for each non-zero element:

$$\text{GFLOPS} = \frac{2 \times \text{NNZ}}{T_{avg} \times 10^9} \tag{1}$$

- Effective Bandwidth (BW) (GB/s): As an intensely *memory bound* operation, the actual bandwidth changes depending on the structural footprint of the analyzed format. For CSR:

$$\text{BW}_{CSR} = \frac{S_{int}(M + 1 + \text{nnz}) + S_{flt}(\text{nnz} + N + M)}{T_{avg} \times 10^9} \tag{2}$$

  For COO, which does not compress row indices, the matrix footprint grows, and so does the expression in bytes:

$$\text{BW}_{COO} = \frac{S_{int}(2 \times \text{nnz}) + S_{flt}(\text{nnz} + N + M)}{T_{avg} \times 10^9} \tag{3}$$

  where $S_{int} = S_{flt} = 4$ bytes.
- Cache Statistics (CPU): The targeted profiling of the hierarchical cache system on the CPU was conducted using the Valgrind tool. Through dedicated formulas, such as

$$\text{D1 Miss (\%)} = \left(\frac{\text{D1 Misses}}{\text{Total D1 Accesses}} \right) \times 100 \tag{4}$$

  it is possible to quantify access efficiency and visualize the theoretical bandwidth collapse due to memory collisions.

E. Correctness and Validation

Due to the non-deterministic nature of parallel execution and the non-associativity of floating-point arithmetic (`Float32`, IEEE 754 standard), a simple syntactic check on bytes would fail the functional verification.

The comparison between the results generated by the GPU and the sequential *baseline* on the CPU therefore takes place through a dedicated method (`validate_results()`), based on a rigorous numerical tolerance (with absolute and relative thresholds). If the divergence between an element computed by the CPU and that outputted by the GPU exceeds this threshold, the experiment declares a FAIL state. This standard ensures that the different architectural versions applied do not compromise the mathematical integrity of the result.

III. Dataset

The experimental evaluation is conducted on a selection of 10 sparse matrices taken from the *SuiteSparse Matrix Collection*, following the benchmark suite proposed by Chu et al. [1]. These datasets were chosen to represent a wide variety of application domains and structural characteristics.

TABLE II
STRUCTURAL CHARACTERISTICS OF THE SELECTED SUITESPARSE MATRICES

Matrix Name	Rows (M)	Columns (N)	NNZ	Mean NNZ/R	Std. Dev. Rows
ASIC_680ks	682,712	682,712	1,693,767	2.48	4.16
bone010	986,703	986,703	47,851,783	48.50	7.62
boyd2	466,316	466,316	1,500,397	3.22	194.91
FullChip	2,987,012	2,987,012	26,621,983	8.91	1,806.80
Ga41As41H72	268,096	268,096	18,488,476	68.96	105.39
ldoor	952,203	952,203	42,493,817	44.63	14.77
rajat31	4,690,002	4,690,002	20,316,253	4.33	1.11
Rucci1	1,977,885	109,900	7,791,168	3.94	0.34
Si41Ge41H72	185,639	185,639	15,011,265	80.86	126.97
webbase-1M	1,000,005	1,000,005	3,105,536	3.11	25.35

IV. Results

Plots/tables for runtime and FLOPS/GFLOP/s; optional memory/cache indicators. *(Note: Visual data presentation only; formulas are established in Methodology).*

- **Throughput & Runtime:** Include bar charts comparing GFLOP/s across the 10 matrices for Base CSR, Base COO, Opt CSR, and cuSPARSE. Include kernel-time plots displaying the measured temporal variability (e.g., standard deviation or min-max bounds).
- **Bandwidth Plots:** Create Effective Bandwidth (GB/s) charts. Add a horizontal line representing the Theoretical Peak Bandwidth (e.g., $\sim$933.1 GB/s) to visually demonstrate how close the implementations are to being memory-bound.
- **Memory/Cache Indicators:** Provide charts or tables (using data from `valgrind`) showing cache hit/miss behavior and global memory accesses.
- Nonostante le ottimizzazioni del kernel, il TTS rimane dominato dal caricamento I/O per le matrici meno dense
- validazione dei risultati con errore etc

V. Discussion

Interpret the results in terms of format properties, matrix structure, and memory behavior. Connect the measured behavior to algorithmic techniques and matrix characteristics.

- **Load Balancing & Matrix Structure:** Do not simply state "kernel X is faster". Explain *why*. Discuss how row-length regularity and variance affect performance. For instance, precisely explain under which structural conditions (e.g., highly irregular matrices) CSR-Vector outperforms CSR-Base by mitigating load imbalance.
- **Memory Behavior & SM Utilization:** Address how close the results are to the memory bound and what evidence supports this. Use the `valgrind` profile cache for cpu.
- **The cuSPARSE Advantage:** Analyze the performance gap with cuSPARSE, theorizing its use of adaptive load-balancing techniques, dynamic partitioning, or undocumented heuristics tailored to specific matrix regimes.

VI. Conclusion

Main lessons learned, limitations, and practical takeaways.

- **Main Lessons Learned:** Summarize which observations are intrinsic to the mathematical structure of the matrix itself, and which are artifacts of specific implementation choices.
- **Practical Takeaways:** Clearly state the optimal use-cases. Explain exactly under which matrix characteristics (e.g., low NNZ/row, high variance) and memory-access conditions a specific kernel or format ceases to be advantageous and another should be preferred.
- **Limitations:** Briefly mention study limitations (e.g., exclusively testing `Float32` arithmetic or specific Ampere architecture constraints).

References

[1] G. Chu, Y. He, L. Dong, Z. Ding, D. Chen, H. Bai, X. Wang, and C. Hu, "Efficient Algorithm Design of Optimizing SpMV on GPU," in *Proc. 32nd Int. Symp. High-Perform. Parallel Distrib. Comput. (HPDC '23)*, 2023, pp. 243–256. doi: 10.1145/3588195.3593002.